

Vision-Guided Autonomous Landing of a Parrot Mambo Minidrone on a Moving Ground Platform

Jatin Satyam

School for Engineering of Matter, Transport and Energy
Ira A. Fulton Schools of Engineering, Arizona State
University, Tempe, AZ
Email: jsatyam@asu.edu

Abstract—This work describes a vision-driven pipeline, built end to end in Simulink, that takes a Parrot Mambo minidrone from line following to tracking a moving ground platform and finally to settling onto that platform without any external positioning aid. Frames from the onboard downward camera are converted from YUV to RGB, thresholded to isolate the target, cleaned by median filtering, and reduced by blob analysis to the centroid of the dominant region. The offset between that centroid and the image center yields a two-axis error signal, which a Stateflow chart converts into corrected position references. Descent onto the moving platform is executed as a sequence of small altitude steps, each gated on the velocity error, so the drone never drops toward empty floor beside a moving target. In simulation testing with tuned gains the drone held position within roughly five centimeters of the moving platform, and the deployed flight lands on the platform while it travels along its track, in a single continuous take.

Index Terms—Parrot Mambo, Simulink, moving platform landing, blob analysis, Stateflow, visual servoing, autonomous landing.

I. INTRODUCTION

Microdrones occupy an interesting niche in indoor robotics: small enough to be safe around people, light enough for a small battery, and increasingly capable thanks to integrated sensing. The goal here was a vision-only navigation stack using nothing more than the Mambo's downward camera and classical image processing inside a closed control loop, developed incrementally: first stable line following, then tracking of a moving platform at constant altitude, and finally a committed landing on that platform in motion.

The Parrot Mambo carries the sensor suite the whole series relies on: an ultrasound sensor timing pulses to the floor for altitude, a pressure sensor for altitude changes, an IMU providing linear acceleration and angular rates, and a downward-facing camera whose optical flow contributes horizontal velocity estimates. Four motors in an X configuration are commanded through motor mixing: collective thrust, a differential between the clockwise and counter-clockwise pairs for yaw, and side-to-side speed imbalances for pitch and roll. The airframe is underactuated, with six degrees of freedom and only four inputs, so every lateral motion begins as a deliberate tilt of the thrust

vector.

II. SYSTEM ARCHITECTURE

All flight code was developed in Simulink against the Parrot Minidrone Support Package and deployed to the physical airframe, keeping the model structure the support package expects: an estimator consuming the sensor buses, a flight control system, and reference logic that the author modified per task. Structuring the work as small reusable subsystems made iteration and debugging on hardware considerably less painful.

Each functional concern lives in its own subsystem: image acquisition and YUV-to-RGB conversion; tracking, with the color threshold, median smoothing, blob analysis, and centroid-error computation; path planning, where a Stateflow chart converts errors into position commands; and control integration, where vision corrections and the onboard estimator's state are merged into the reference bus the navigation system consumes.

III. METHOD

A. Vision front end

A lightweight color filter checks each pixel against channel bounds and returns a binary mask of the target. Median filtering removes speckle while preserving edge geometry, and blob analysis retains the dominant region, returning its centroid and bounding box for both control and the visual debug viewers. The centroid is compared against the image center to produce X and Y error signals.

B. Tracking a moving platform

With the same front end retargeted at the platform's signature color, the centroid offset is converted into lateral and longitudinal velocity commands through a small gain, deliberately kept low to keep the loop calm. As the ground robot drives, the error grows in its direction of motion and the drone follows.

C. Descent logic

Descent is a sequence of small altitude steps rather than a continuous push downward. Each step checks the velocity error before committing, so the drone will not drop further until it has caught up with the platform laterally. After contact, the motors ramp down gradually to prevent bounce or tilt.

IV. RESULTS AND DISCUSSION

In simulation testing with tuned gains, the drone held position within roughly five centimeters of the moving platform, adjusting quickly to lateral and longitudinal shifts, and the stepwise descent produced controlled, repeatable approach behavior. Temporary misalignments appeared under sharp acceleration or braking of the platform, and were largely resolved by synchronizing the velocity command logic more carefully with the latest vision update. In the deployed flight, the platform travels along its track while the drone tracks it from above and commits to the landing, touching down on the moving target in one continuous take.



Fig. 1. Landing approach: the platform travels along its track while the drone tracks it from above.

V. CHALLENGES

- Lighting variability: brightness shifts and shadows occasionally split the blob or erased it for a frame, causing short bursts of misalignment.
- Fixed color thresholds do not generalize across scene changes; adaptive thresholding is the clear next step.
- Latency between a vision update and a velocity setpoint taking effect, visible as small oscillations when the platform accelerated or braked sharply.

VI. CONCLUSION

The project demonstrates vision-based feedback carrying a minidrone through line following, moving-target tracking, and a committed landing on a platform in motion. The closed-loop visual pipeline handled both the static and dynamic targets, and the remaining sensitivities around lighting and threshold

calibration chart the path toward a deployable system: adaptive thresholding, further real-world flight testing, and tighter treatment of vision-control timing.

APPENDIX: MATLAB FUNCTION CODE

The following MATLAB Function blocks are the author's custom code from the deployed Simulink model, reproduced verbatim.

Listing 1. *green_color*: HSV threshold isolating the target

```
function BW = green_color(R,G,B)

% HSV
I = cat(3,R,G,B);

I = rgb2hsv(I);

% Define thresholds for channel 1 based on histogram settings
channel1Min = 0.832;
channel1Max = 0.057;

% Define thresholds for channel 2 based on histogram settings
channel2Min = 0.347;
channel2Max = 1.000;

% Define thresholds for channel 3 based on histogram settings
channel3Min = 0.483;
channel3Max = 1.000;

sliderBW = ( (I(:,:,1) >= channel1Min) | (I(:,:,1) <= channel1Max) ) & ...
    (I(:,:,2) >= channel2Min) & (I(:,:,2) <= channel2Max) & ...
    (I(:,:,3) >= channel3Min) & (I(:,:,3) <= channel3Max);
BW = sliderBW;

end
```

Listing 2. *Earth-to-body frame error transformation*

```
function xy_err_body = fcn(yaw,xy_err_earth)

transformation = [cos(yaw), sin(yaw); -sin(yaw), cos(yaw)];
xy_err_body = transformation * xy_err_earth;

end
```

ACKNOWLEDGMENT

The author completed this work as part of the RAS 546 curriculum in the MS Robotics and Autonomous Systems program at Arizona State University, and thanks the course staff for guidance on the Parrot Mambo platform, flight-arena logistics, and safe flight operations.

REFERENCES

- [1] MathWorks, “Simulink Support Package for Parrot Minidrones,” MathWorks documentation, 2025.
- [2] MathWorks, “Getting Started with Simulink for Parrot Minidrone Hover,” MathWorks examples, 2025.
- [3] B. Douglas, “Drone Simulation and Control” video series, MathWorks / Control Systems in Practice.
- [4] MathWorks, “MathWorks Minidrone Competition,” competition guidelines and arena specification, 2025.